

Tecnatura Superior en Desarrollo de Software

Módulo 06 — Validación de Modelos: `IModelValidatorHub<T>`

Actividad Práctica — CU-CLI-001: Alta de Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

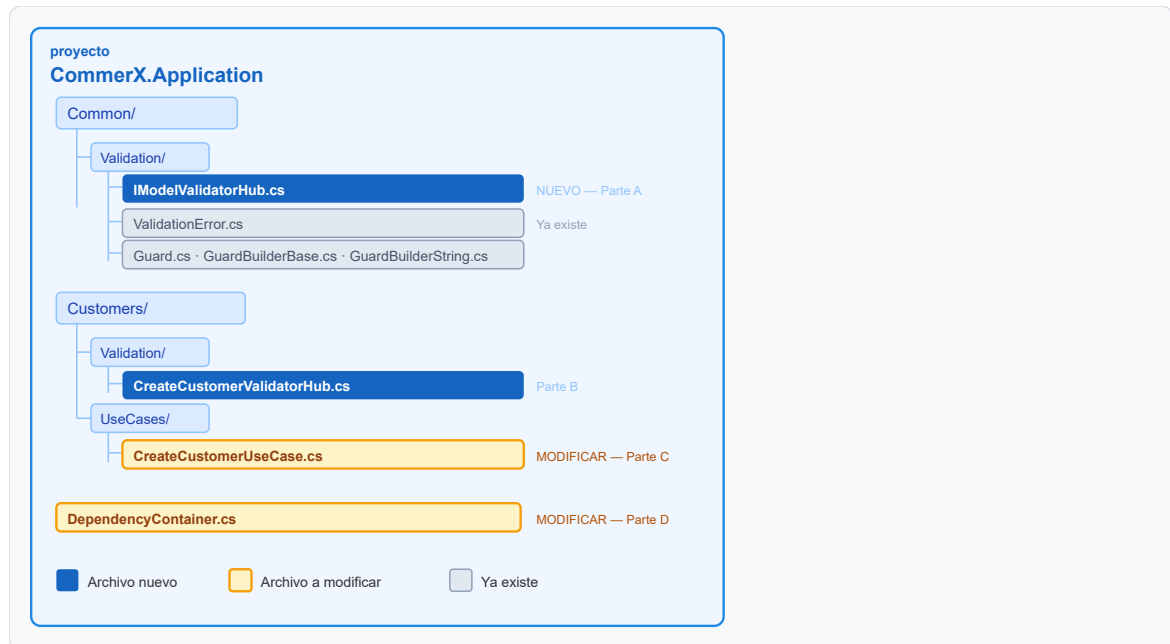
Contexto de la actividad

En el **Módulo 05** integraste los Guards directamente en `CreateCustomerUseCase`: un Guard por campo, todos concatenados dentro del método `ExecuteAsync`. Esa solución funciona, pero mezcla la responsabilidad de validación con la orquestación del flujo de negocio.

En esta actividad extraés esa lógica a una clase dedicada: `CreateCustomerValidatorHub`. El UseCase queda reducido a una sola línea de validación y delega completamente en el hub.

Objetivo: Crear `IModelValidatorHub<T>`, implementar `CreateCustomerValidatorHub`, refactorizar `CreateCustomerUseCase` para usar el hub y registrar todo en `DependencyContainer.cs`.

Archivos involucrados



Parte A — Crear `IModelValidatorHub<T>`

La interfaz define el contrato genérico de validación. Vive en `Common/Validation/` porque es compartida por todos los dominios de la aplicación. Un solo método: dado un modelo de tipo `TModel`, devolver todos los errores encontrados.

Ruta: `CommerX.Application / Common / Validation / IModelValidatorHub.cs`

```

1  // CommerX.Application/Common/Validation/IModelValidatorHub.cs
2  namespace CommerX.Application.Common.Validation;
3
4  // puerto de validación — un método, una responsabilidad
5  public interface IModelValidatorHub<TModel>
6  {
7      // valida el modelo y devuelve todos los errores encontrados
8      // si la colección está vacía, el modelo es válido
9      IEnumerable<ValidationError> Validate(TModel model);
10 }
```

¿Por qué IEnumerable y no List?

`IEnumerable<T>` es la abstracción más amplia para una secuencia. El llamador decide si necesita materializarla con `.ToList()` o simplemente iterarla. El hub no impone una estructura de colección concreta.

Parte B — Crear `CreateCustomerValidatorHub`

La implementación concreta del hub para `CreateCustomerRequest`. Contiene un Guard por cada campo del DTO y combina todos los errores en una sola secuencia con `.Concat()`. No hay lógica de negocio aquí — solo precondiciones técnicas de formato y presencia.

Ruta: `CommerX.Application / Customers / Validation / CreateCustomerValidatorHub.cs`

```
1  // CommerX.Application/Customers/Validation/CreateCustomerValidatorHub.cs
2  using CommerX.Application.Common.Validation;
3  using CommerX.Application.Customers.DTOs;
4
5  namespace CommerX.Application.Customers.Validation;
6
7  // hub concreto para CreateCustomerRequest
8  public class CreateCustomerValidatorHub
9      : IModelValidatorHub<CreateCustomerRequest>
10 {
11     public IEnumerable<ValidationError> Validate(CreateCustomerRequest request)
12     {
13         // un Guard por campo — cada uno acumula sus propios errores
14         var gFirstName = Guard.Against(request.FirstName, "FirstName")
15             .NotNullOrEmpty().MinLength(2).MaxLength(100);
16
17         var gLastName = Guard.Against(request.LastName, "LastName")
18             .NotNullOrEmpty().MinLength(2).MaxLength(100);
19
20         var gDocument = Guard.Against(request.Document, "Document")
21             .NotNullOrEmpty().MinLength(7).MaxLength(8);
22
23         var gEmail = Guard.Against(request.Email, "Email")
24             .NotNullOrEmpty().InvalidEmail();
25
26         var gPhone = Guard.Against(request.Phone, "Phone")
27             .NotNullOrEmpty().MaxLength(20);
```

```
28
29     var gAddress = Guard.Against(request.Address, "Address")
30         .NotNullOrEmpty().MaxLength(200);
31
32     // combina todos los errores en una sola secuencia y la devuelve
33     return gFirstName.Errors
34         .Concat(gLastName.Errors)
35         .Concat(gDocument.Errors)
36         .Concat(gEmail.Errors)
37         .Concat(gPhone.Errors)
38         .Concat(gAddress.Errors);
39 }
40 }
```

Regla clave: El hub devuelve la secuencia *sin materializar* (sin llamar a `.ToList()`). El UseCase la materializa una sola vez cuando necesita verificar `errors.Count`.

Parte C — Refactorizar `CreateCustomerUseCase`

El UseCase del Módulo 05 contenía los Guards directamente en `ExecuteAsync`. Ahora se refactoriza para recibir el hub por inyección de dependencias y delegar en él toda la validación. El resultado es un método más limpio y con una sola responsabilidad claramente delimitada.

Ruta: `CommerX.Application / Customers / UseCases / CreateCustomerUseCase.cs`

```
1  // CommerX.Application/Customers/UseCases/CreateCustomerUseCase.cs
2  using CommerX.Application.Common.Validation;
3  using CommerX.Application.Customers.DTOs;
4  using CommerX.Application.Customers.Ports;
5  using CommerX.Application.Customers.Validation;
6  using CommerX.Domain.Common.Exceptions;
7  using CommerX.Domain.Customers.Entities;
8  using CommerX.Domain.Customers.Repositories;
9
10 namespace CommerX.Application.Customers.UseCases;
11
12 // sealed: su comportamiento es definitivo – no puede heredarse
13 public sealed class CreateCustomerUseCase : ICreateCustomerInputPort
14 {
```

```
15 // repositorio inyectado – contrato definido en Domain
16 private readonly ICustomerRepository _repository;
17 // puerto de salida inyectado – notifica el resultado al llamador
18 private readonly ICreateCustomerOutputPort _outputPort;
19 // hub de validación – valida las precondiciones técnicas del DTO
20 private readonly IModelValidatorHub<CreateCustomerRequest> _validator;
21
22 public CreateCustomerUseCase(
23     ICustomerRepository repository,
24     ICreateCustomerOutputPort outputPort,
25     IModelValidatorHub<CreateCustomerRequest> validator)
26 {
27     _repository = repository;
28     _outputPort = outputPort;
29     _validator = validator;
30 }
31
32 public async Task ExecuteAsync(CreateCustomerRequest request)
33 {
34     // PASO 1 – Guard Hub: precondiciones técnicas del DTO
35     var errors = _validator.Validate(request).ToList();
36     if (errors.Count > 0)
37     {
38         // notifica todos los errores juntos – el dominio no es invocado
39         await _outputPort.ValidationErrorsAsync(errors);
40         return;
41     }
42
43     try
44     {
45         // verificamos que no exista un cliente con el mismo documento
46         Customer? existing = await _repository.FindByDocumentAsync(request.Document);
47
48         if (existing is not null)
49         {
50             // notificamos la duplicidad y detenemos la ejecución
51             await _outputPort.HandleDuplicateAsync(request.Document);
52             return;
53         }
54
55         // PASO 2 – Dominio: el dominio valida las reglas – puede lanzar DomainException
56         var customer = Customer.Create(
57             request.FirstName,
58             request.LastName,
59             request.Document,
60             request.Email,
```

```
61         request.Phone,
62         request.Address,
63         request.BirthDate
64     );
65
66     // PASO 3 – Persistencia: persistimos el nuevo cliente
67     await _repository.AddAsync(customer);
68
69     // PASO 4 – Notificación: construimos el DTO de respuesta y notificam
os el éxito
70     var response = new CreateCustomerResponse()
71     {
72         CustomerId = customer.Id
73     };
74
75     await _outputPort.HandleSuccessAsync(response);
76 }
77 catch (DomainException ex)
78 {
79     // cualquier excepción de dominio se redirige al puerto de salida – n
unca se relanza
80     await _outputPort.HandleValidationErrorAsync(ex.Message);
81 }
82 }
83 }
```

Cambio clave respecto al Módulo 05: Los Guards individuales por campo desaparecieron del UseCase. El campo `_validator` y la llamada a `_validator.Validate(request).ToList()` en la línea 35 reemplazan todo ese bloque. El resto del flujo (unicidad, dominio, persistencia, notificación) queda intacto.

Parte D — Registrar el hub en `DependencyContainer.cs`

El UseCase depende de `IModelValidatorHub<CreateCustomerRequest>` — una interfaz. Para que el contenedor de DI pueda resolver esa dependencia, hay que indicarle qué clase concreta inyectar. El registro se agrega en el método de extensión existente de la capa Application.

Ruta: `CommerX.Application / DependencyContainer.cs`

```
1 // CommerX.Application/DependencyContainer.cs
```

```
2 using CommerX.Application.Common.Validation;
3 using CommerX.Application.Customers.DTOs;
4 using CommerX.Application.Customers.Ports;
5 using CommerX.Application.Customers.UseCases;
6 using CommerX.Application.Customers.Validation;
7
8 // namespace truco – evita agregar using en Program.cs
9 namespace Microsoft.Extensions.DependencyInjection;
10
11 public static class DependencyContainer
12 {
13     public static IServiceCollection AddCustomerUseCases(
14         this IServiceCollection services)
15     {
16         // UseCase – Scoped: una instancia por petición HTTP
17         services.AddScoped<ICreateCustomerInputPort, CreateCustomerUseCase>();
18
19         // hub – Scoped: mismo ciclo de vida que el UseCase
20         services.AddScoped<
21             IModelValidatorHub<CreateCustomerRequest>,
22             CreateCustomerValidatorHub>();
23
24         return services;
25     }
26 }
```

¿Por qué Scoped y no Singleton?

El hub no tiene estado propio — cada llamada a `Validate()` trabaja con el request que recibe. Sin embargo se registra como `Scoped` para mantener coherencia de ciclo de vida con el UseCase: ambos nacen y mueren en el mismo contexto de operación. Si el hub en el futuro necesitara una dependencia Scoped (poco probable), ya estaría correctamente configurado.

Parte E — Checklist de verificación

Antes de dar por completada la actividad, verificá que cada punto esté presente y correctamente implementado en tu proyecto.

N.º	Archivo	Qué verificar
1	<code>IModelValidatorHub.cs</code>	Interfaz genérica en <code>Common/Validation/</code> . Un solo método <code>Validate(TModel)</code> que devuelve <code>IEnumerable<ValidationError></code> .
2	<code>CreateCustomerValidatorHub.cs</code>	Implementa <code>IModelValidatorHub<CreateCustomerRequest></code> . Un Guard por campo del DTO. Devuelve la secuencia con <code>.Concat()</code> sin llamar a <code>.ToList()</code> .
3	<code>CreateCustomerUseCase.cs</code>	Constructor recibe <code>IModelValidatorHub<CreateCustomerRequest></code> como tercer parámetro. <code>ExecuteAsync</code> llama a <code>_validator.Validate(request).ToList()</code> antes de cualquier otra operación. No hay Guards individuales en el UseCase.
4	<code>DependencyContainer.cs</code>	<code>AddScoped<IModelValidatorHub<CreateCustomerRequest>, CreateCustomerValidatorHub>()</code> registrado en <code>AddCustomerUseCases()</code> .
5	Compilación	El proyecto <code>CommerX.Application</code> compila sin errores. No hay referencias a Guards individuales dentro de <code>ExecuteAsync</code> .

Parte F — Preguntas de reflexión

- ¿Por qué `IModelValidatorHub<T>` es un puerto y no una clase concreta?
¿Qué ventaja tiene que el UseCase dependa de la interfaz y no de `CreateCustomerValidatorHub` directamente?
- ¿Qué sucede si olvidas registrar el hub en `DependencyContainer.cs`?
¿En qué momento falla? ¿En tiempo de compilación o en tiempo de ejecución? ¿Cuál sería el mensaje de error?
- ¿Por qué el hub devuelve `IEnumerable` y el UseCase llama a `.ToList()`?
¿Qué pasaría si el hub llamara a `.ToList()` internamente y el UseCase también lo hiciera?
- ¿Cuál es la diferencia entre validación técnica (hub) y validación de dominio (Value Object)?
Dá un ejemplo de cada una usando los campos de `CreateCustomerRequest`.

5. **¿Dónde se verifica la unicidad del documento y por qué no en el hub?**
¿Qué principio de diseño justifica esa decisión?
6. **Si en el futuro se agregara el campo `SecondLastName` al DTO, ¿qué archivos habría que modificar?**
¿El UseCase debería cambiar? ¿Por qué sí o por qué no?

Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Tecnicatura Superior en Desarrollo de Software |
Asignatura: Programación II | Tema: Módulo 06 — Validación de Modelos — Actividad Práctica | Versión: 2026